

Item 01

- **Dificuldade:** Fácil
 - **Conhecimento:** 1.1. Documentação de APIs
 - **Referência:** Aula 02
-

Capacidade: CT01 - Reconhecer as especificações dos serviços disponíveis no servidor

Contexto: Um desenvolvedor precisa configurar os metadados de uma requisição HTTP para informar ao servidor que o corpo (body) da mensagem que está sendo transmitida contém dados formatados em JSON (*application/json*).

Comando: Identifique o cabeçalho (*header*) de requisição HTTP utilizado para especificar o formato do corpo de dados transmitido.

Alternativas:

- ☐ A) User-Agent
 - ☐ B) Accept-Encoding
 - ☐ C) Content-Type
 - ☐ D) Referer
 - ☐ E) Host
-

Gabarito: C

Justificativa: O cabeçalho *Content-Type* informa ao servidor (ou ao cliente) qual é o tipo de mídia ou formato (tipo MIME) do corpo da mensagem (ex: *application/json*).

Distratores:

- A) O *User-Agent* é o cabeçalho que identifica o navegador, sua versão e o sistema operacional do cliente.
 - B) O *Accept-Encoding* informa ao servidor quais algoritmos de compressão de dados (como gzip) o navegador suporta.
 - C) Alternativa correta.
 - D) O *Referer* registra o endereço da página web anterior que continha o link que gerou a requisição atual.
 - E) O *Host* especifica o nome do domínio do servidor e a porta para a qual a requisição está sendo enviada.
-

Item 02

- **Dificuldade:** Fácil
 - **Conhecimento:** 1.2. Operações (CRUD)
 - **Referência:** Aula 03
-

Capacidade: CT04 - Reconhecer as necessidades de utilização dos serviços do servidor

Contexto: Ao executar chamadas assíncronas em JavaScript, uma *Promise* é gerada como um comprovante da operação. A aplicação precisa monitorar o ciclo de vida dessa operação até a chegada do resultado.

Comando: Identifique o estado de uma *Promise* que indica que a operação assíncrona ainda não foi concluída e está aguardando o resultado.

Alternativas:

- ☐ A) fulfilled
- ☐ B) rejected
- ☐ C) pending
- ☐ D) resolved

☐ E) done

Gabarito: C

Justificativa: O estado **pending** (pendente) indica que a operação assíncrona foi iniciada, mas ainda não terminou de ser processada.

Distratores:

- A) O estado **fulfilled** (realizada) indica que a operação foi concluída com sucesso e o resultado está disponível.
 - B) O estado **rejected** (rejeitada) indica que ocorreu uma falha ou erro durante o processamento da operação.
 - C) Alternativa correta.
 - D) O termo *resolved* refere-se ao ato de concluir uma promessa, mas os três estados formais de uma Promise são pending, fulfilled e rejected.
 - E) O termo *done* não representa um estado formal da especificação de Promises em JavaScript.
-

Item 03

- **Dificuldade:** Fácil
 - **Conhecimento:** 1.3. Tratamento de mensagens do server-side
 - **Referência:** Aula 03
-

Capacidade: CT05 - Aplicar tratamento de falhas nas mensagens do servidor

Contexto: Ao utilizar a Fetch API para consultar dados no servidor, o programador deve verificar se a resposta obtida indica sucesso. Ele precisa validar se o código de status retornado pelo servidor está situado na faixa de 200 a 299.

Comando: Indique a propriedade booleana do objeto **Response** retornado pela função **fetch()** utilizada para confirmar se o status da resposta indica sucesso.

Alternativas:

- ☐ A) **response.statusText**
 - ☐ B) **response.ok**
 - ☐ C) **response.body**
 - ☐ D) **response.headers**
 - ☐ E) **response.json**
-

Gabarito: B

Justificativa: A propriedade **response.ok** retorna **true** quando o código HTTP de status da resposta está na faixa de sucesso (entre 200 e 299).

Distratores:

- A) A propriedade **response.statusText** contém a mensagem textual associada ao código de status (ex: "OK" ou "Not Found"), não um valor booleano.
 - B) Alternativa correta.
 - C) A propriedade **response.body** contém o fluxo (stream) do corpo da resposta tratada pelo navegador.
 - D) A propriedade **response.headers** guarda o mapa com os cabeçalhos de resposta enviados pelo servidor.
 - E) O método **response.json()** é uma função assíncrona que converte o corpo da resposta em um objeto JavaScript, e não uma propriedade booleana de validação de status.
-

Item 04

- **Dificuldade:** Fácil
 - **Conhecimento:** 1.2. Operações (CRUD)
 - **Referência:** Aula 03
-

Capacidade: CT04 - Reconhecer as necessidades de utilização dos serviços do servidor

Contexto: Um desenvolvedor estruturou uma função assíncrona para consultar o status de um pedido em uma API. Ele precisa utilizar a palavra-chave que pausa a execução interna da função até que a promessa de rede seja resolvida, sem travar a interface do usuário.

Comando: Identifique a palavra-chave utilizada exclusivamente dentro de funções declaradas como `async` para pausar a execução da função até a resolução de uma `Promise`.

Alternativas:

- ☐ A) `fetch`
 - ☐ B) `then`
 - ☐ C) `await`
 - ☐ D) `catch`
 - ☐ E) `finally`
-

Gabarito: C

Justificativa: A palavra-chave `await` só pode ser usada dentro de funções `async` e pausa a execução interna daquela função até que a `Promise` seja resolvida.

Distratores:

- A) O `fetch` é a função do navegador utilizada para realizar requisições HTTP, retornando uma `Promise`.
 - B) O `.then()` é um método de encadeamento de Promises na sintaxe tradicional, não a palavra-chave da estrutura `async/await`.
 - C) Alternativa correta.
 - D) O `catch` é utilizado para o tratamento e captura de exceções/erros na estrutura `try/catch` ou no encadeamento de Promises.
 - E) O `finally` define um bloco de código executado obrigatoriamente ao término de uma operação, independente de sucesso ou falha.
-

Item 05

- **Dificuldade:** Fácil
 - **Conhecimento:** 1.2. Operações (CRUD)
 - **Referência:** Aula 04
-

Capacidade: CT04 - Reconhecer as necessidades de utilização dos serviços do servidor

Contexto: Para testar a gravação e a persistência de dados em um protótipo de painel web sem violar as regras de segurança do navegador que impedem a alteração de arquivos locais no disco rígido, o desenvolvedor optou por utilizar a memória interna do próprio navegador.

Comando: Indique o recurso de armazenamento web do navegador utilizado para guardar dados em formato de texto de maneira permanente no lado do cliente.

Alternativas:

- ☐ A) Call Stack
- ☐ B) Live Server
- ☐ C) `localStorage`
- ☐ D) Event Loop

☐ E) Swagger UI

Gabarito: C

Justificativa: O **localStorage** é uma chave de armazenamento interno e permanente presente em navegadores modernos, utilizada para guardar dados no formato de texto JSON sem expirar ao fechar a aba.

Distratores:

- A) A Call Stack (Pilha de Execução) é a estrutura da memória do motor JavaScript onde as funções em execução são empilhadas temporariamente.
 - B) O Live Server é um servidor local de desenvolvimento fornecido como extensão do editor de código, não um recurso de memória do navegador.
 - C) Alternativa correta.
 - D) O Event Loop é o mecanismo do navegador encarregado de gerenciar a entrega de tarefas assíncronas para a pilha de execução.
 - E) O Swagger UI é uma interface gráfica de documentação técnica para APIs RESTful.
-

Item 06

- **Dificuldade:** Fácil
 - **Conhecimento:** 1.2. Operações (CRUD)
 - **Referência:** Aula 04
-

Capacidade: CT04 - Reconhecer as necessidades de utilização dos serviços do servidor

Contexto: Antes de salvar a lista de objetos de encomendas na memória do **localStorage**, o programador precisa converter essa estrutura de dados do JavaScript em uma string de texto puro.

Comando: Identifique o método do objeto **JSON** utilizado para converter objetos ou arrays do JavaScript em uma string formatada em texto.

Alternativas:

- ☐ A) **JSON.parse()**
 - ☐ B) **JSON.stringify()**
 - ☐ C) **JSON.convert()**
 - ☐ D) **JSON.toText()**
 - ☐ E) **JSON.serialize()**
-

Gabarito: B

Justificativa: O método **JSON.stringify()** pega um objeto ou array do JavaScript e o transforma em uma string no formato de texto JSON, permitindo seu armazenamento ou envio via rede.

Distratores:

- A) O método **JSON.parse()** realiza a operação oposta, convertendo um texto JSON de volta para um objeto/array JavaScript.
 - B) Alternativa correta.
 - C) **JSON.convert()** é uma denominação inexistente na API nativa do objeto JSON em JavaScript.
 - D) **JSON.toText()** não faz parte dos métodos do objeto global JSON no JavaScript.
 - E) **JSON.serialize()** não existe nativamente no JavaScript (a serialização é feita via **JSON.stringify()**).
-

Item 07

- **Dificuldade:** Fácil
- **Conhecimento:** 1.2. Operações (CRUD)
- **Referência:** Aula 04

Capacidade: CT04 - Reconhecer as necessidades de utilização dos serviços do servidor

Contexto: Ao recuperar o texto armazenado no `localStorage`, a aplicação web precisa converter a string contendo os dados das entregas de volta para uma lista de objetos manipulável pelo código JavaScript.

Comando: Indique o método do objeto `JSON` empregado para converter uma string de texto formatada em JSON de volta para objetos ou arrays do JavaScript.

Alternativas:

- ☐ A) `JSON.stringify()`
- ☐ B) `JSON.parse()`
- ☐ C) `JSON.object()`
- ☐ D) `JSON.decode()`
- ☐ E) `JSON.read()`

Gabarito: B

Justificativa: O método `JSON.parse()` analisa uma string de texto formatada em JSON e a restaura em estruturas nativas (objetos ou listas) do JavaScript.

Distratores:

- A) O método `JSON.stringify()` executa a ação inversa, convertendo um objeto JavaScript em texto.
- B) Alternativa correta.
- C) `JSON.object()` não é um método nativo da linguagem JavaScript.
- D) `JSON.decode()` não existe no objeto global `JSON` em JS (embora termos parecidos existam em outras linguagens como PHP).
- E) `JSON.read()` é uma denominação fictícia não pertencente ao ecossistema padrão do JavaScript.

Item 08

- **Dificuldade:** Fácil
- **Conhecimento:** 1.2. Operações (CRUD)
- **Referência:** Aula 05

Capacidade: CT03 - Aplicar boas práticas relativas à segurança da informação

Contexto: Um desenvolvedor júnior está organizando o código de um módulo utilitário e adotando as boas práticas do JavaScript moderno (ES6+) para declarar identificadores com escopo de bloco que não devem sofrer reatribuições acidentais de valor.

Comando: Identifique a palavra-chave recomendada como regra de ouro padrão para declarar variáveis com escopo de bloco cujo valor não deve ser reatribuído após a inicialização.

Alternativas:

- ☐ A) `var`
- ☐ B) `let`
- ☐ C) `const`
- ☐ D) `function`
- ☐ E) `global`

Gabarito: C

Justificativa: No JavaScript moderno (ES6+), a recomendação padrão é declarar variáveis usando `const`, o que garante o escopo de bloco e impede alterações acidentais por reatribuição de valor.

Distratores:

- A) A palavra-chave **var** possui escopo de função e comportamento imprevisível (*hoisting*), sendo desautorizada nas boas práticas modernas.
 - B) A palavra-chave **let** possui escopo de bloco, mas é utilizada especificamente quando há necessidade de reatribuir o valor da variável futuramente (como em contadores).
 - C) Alternativa correta.
 - D) A palavra-chave **function** é utilizada para declaração de blocos de código executáveis (funções), não para o armazenamento de dados/variáveis.
 - E) O termo **global** não é uma instrução de declaração de variáveis no JavaScript.
-

Item 09

- **Dificuldade:** Fácil
 - **Conhecimento:** 1.2. Operações (CRUD)
 - **Referência:** Aula 05
-

Capacidade: CT04 - Reconhecer as necessidades de utilização dos serviços do servidor

Contexto: Durante a codificação de rotinas de manipulação de dados em uma biblioteca utilitária, o programador utilizou uma sintaxe curta e moderna introduzida no ES6 para definir funções anônimas e callbacks.

Comando: Indique a denominação dada à sintaxe moderna de função em JavaScript caracterizada pelo uso do símbolo de seta (**=>**).

Alternativas:

- ☐ A) IIFE
 - ☐ B) Arrow Function
 - ☐ C) Function Declaration
 - ☐ D) Class Method
 - ☐ E) Generator Function
-

Gabarito: B

Justificativa: As *Arrow Functions* são funções com sintaxe curta introduzidas no ES6, definidas através do operador de seta (**=>**).

Distratores:

- A) IIFE (Immediately Invoked Function Expression) representa uma expressão de função executada imediatamente após sua criação.
 - B) Alternativa correta.
 - C) *Function Declaration* refere-se à declaração tradicional de função utilizando a palavra-chave **function**.
 - D) *Class Method* indica um método pertencente à estrutura de uma classe orientada a objetos.
 - E) *Generator Function* refere-se a funções especiais que podem ter sua execução pausada e retomada usando a palavra-chave **yield**.
-

Item 10

- **Dificuldade:** Fácil
 - **Conhecimento:** 1.2. Operações (CRUD)
 - **Referência:** Aula 05
-

Capacidade: CT03 - Aplicar boas práticas relativas à segurança da informação

Contexto: Em uma aplicação web, a equipe de desenvolvimento criou um arquivo separado chamado `utils.js` para reunir funções genéricas de formatação de datas e máscaras de texto. Essa organização evita a cópia repetida de código em diferentes arquivos do projeto.

Comando: Identifique o princípio de desenvolvimento de software cuja sigla representa o conceito de evitar a duplicação desnecessária de lógica (*Don't Repeat Yourself*).

Alternativas:

- ☐ A) SPA
 - ☐ B) DRY
 - ☐ C) CRUD
 - ☐ D) JWT
 - ☐ E) CORS
-

Gabarito: B

Justificativa: O princípio DRY (*Don't Repeat Yourself* — Não se Repita) orienta que cada porção de conhecimento ou lógica de software deve ter uma representação única no sistema, evitando a duplicação de código.

Distratores:

- A) SPA (Single Page Application) refere-se a uma arquitetura de aplicação web em página única sem recarregamento de tela.
 - B) Alternativa correta.
 - C) CRUD é o acrônimo para as quatro operações básicas de dados: Create, Read, Update e Delete.
 - D) JWT (JSON Web Token) é um padrão para autenticação e troca segura de informações entre partes.
 - E) CORS (Cross-Origin Resource Sharing) é um mecanismo de segurança de rede que permite a requisição de recursos em domínios distintos.
-